**ESCUELA DE CIENCIAS APLICADAS E INGENIERÍA**

Departamento de Ingeniería

# Trabajo Practico 01

## Ejercicio 2: Coloracion de Grafos con $k$ Colores (Backtracking)

**Asignatura:** Analisis y Diseño de Algoritmos

**Profesor:** Carlos Alberto Álvarez Henao

**Integrantes:** Andrés Felipe Vélez Álvarez
Sebastian Salazar Henao
Nathalia Valentina Cardoza Azuaje

**Fecha:** 5 de abril de 2026

Universidad EAFIT

# Contents


**1 Descripcion Teorica del Problema** **2**

**2 Enfoques de Solucion** **2**
2.1 Backtracking . . . . . 2
2.2 Fuerza Bruta . . . . . 3
2.3 Comparacion de Enfoques . . . . . 3

**3 Pseudocodigo** **3**
3.1 Backtracking . . . . . 3
3.2 Fuerza Bruta . . . . . 4

**4 Implementacion en C++** **4**

**5 Ejemplos de Entrada y Salida** **5**
5.1 Ejemplo 1 — Ciclo C4 (n=4, k=3) . . . . . 5
5.2 Ejemplo 2 — Grafo completo K4 (n=4, k=4) . . . . . 6
5.3 Ejemplo 3 — Ciclo C7 con cuerdas (n=7, k=3) — Sin solucion . . . . . 7
5.4 Ejemplo 4 — Bipartito K2,3 (n=5, k=2) . . . . . 8

**6 Comparacion de Tiempos de Ejecucion** **9**

**7 Analisis de Complejidad** **10**
7.1 Complejidad Temporal . . . . . 10
7.1.1 Fuerza Bruta . . . . . 10
7.1.2 Backtracking . . . . . 10
7.2 Complejidad Espacial . . . . . 10

**8 Respuestas a las Preguntas Guia** **10**
8.1 ¿Cuantos nodos explora el Backtracking vs la Fuerza Bruta? . . . . . 10
8.2 ¿Que tipo de grafo maximiza o minimiza la poda? . . . . . 11
8.3 ¿Que cambio hacer para saber si existe al menos una k-coloracion? . . . . . 11
8.4 Relacion entre la k-coloracion y el Numero Cromatico . . . . . 11

**9 Conclusiones** **12**

# 1 Descripcion Teorica del Problema

Dado un grafo no dirigido $G = (V, E)$ con $n$ vertices y un entero $k$, la **$k$-coloracion valida** es una asignacion de colores $\{1, \ldots, k\}$ a los vertices tal que ningun par de vertices adyacentes comparte el mismo color:

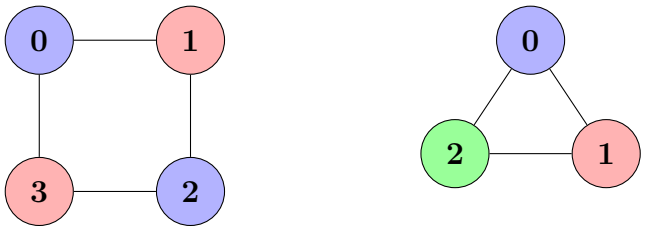
$$\forall\,(u, v) \in E\colon \text{color}[u] \neq \text{color}[v] \tag{1}$$

El problema tiene aplicaciones en horarios de examenes, asignacion de frecuencias de radio y asignacion de registros en compiladores, entre otras.

El **numero cromatico** $\chi(G)$ es el minimo $k$ para el que existe una $k$-coloracion valida. El grafo admite $k$-coloracion si y solo si $\chi(G) \leq k$. Valores canonicos: $\chi(C_{2m}) = 2$, $\chi(C_{2m+1}) = 3$, $\chi(K_n) = n$, y todo grafo planar satisface $\chi \leq 4$ [1].

El grafo se representa mediante una **matriz de adyacencia** $A$ de $n \times n$, donde $A[i][j] = 1$ si existe arista entre $i$ y $j$, y 0 si no. Por ser no dirigido, $A$ es simetrica. Ejemplo para $C_4$:

$$A = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$




$C_4$ con 2 colores $K_3$ con 3 colores

Figure 1: Coloraciones validas: ningun par de vertices adyacentes comparte color.

# 2 Enfoques de Solucion

## 2.1 Backtracking

El **backtracking** (vuelta atras) construye la solucion asignando un color a cada vertice uno por uno. La clave del metodo esta en que, antes de continuar hacia el siguiente vertice, verifica si la asignacion actual genera algun conflicto con los vecinos ya coloreados. Si lo hay, *retrocede* (backtrack) y prueba el siguiente color disponible, sin explorar ese camino que ya se sabe invalido. A esta eliminacion anticipada se le llama **poda**.

Proceso paso a paso:

1. Se intenta asignar un color $c \in \{1, \ldots, k\}$ al vertice $v$.

2. Se verifica que ningun vecino ya coloreado tenga el color $c$ (`es_seguro`).
3. Si hay conflicto: se descarta ese color y se prueba el siguiente — poda inmediata.
4. Si no hay conflicto: se asigna y se avanza al vertice $v + 1$.
5. Cuando $v = n$: todos los vertices estan coloreados; se cuenta como solucion valida.
6. Al regresar de la recursion se deshace la asignacion (backtrack) para explorar otras combinaciones.

### 2.2 Fuerza Bruta

La **fuerza bruta** genera directamente las $k^n$ combinaciones posibles de colores para los $n$ vertices y verifica cada una de ellas. No realiza ninguna poda: aunque los primeros vertices ya presenten un conflicto, el algoritmo sigue enumerando hasta el final antes de descartar esa combinacion.

### 2.3 Comparacion de Enfoques

Table 1: Diferencias entre Backtracking y Fuerza Bruta

| **Aspecto** | **Backtracking** | **Fuerza Bruta** |
|---|---|---|
| Espacio de busqueda | Arbol podado | Todas las $k^n$ combinaciones |
| Poda | Si (conflicto $\Rightarrow$ descarta) | No |
| Completitud | Si | Si |
| Correccion | Si | Si |
| Complejidad peor caso | $O(k^n)$ | $O(k^n \cdot n^2)$ |
| Eficiencia practica | Mucho mejor en grafos densos | Inferior |

## 3 Pseudocodigo

### 3.1 Backtracking

```
leer n, k
leer adj[0..n-1][0..n-1]   // 1 si hay arista, 0 si no
color[0..n-1] = 0           // 0 = sin color asignado
total_soluciones = 0
primera_guardada = falso

// Verifica si el color c es seguro para el vertice v
es_seguro(v, c):
  para cada u en [0..n-1]:
    si adj[v][u] == 1 y color[u] == c:
      retornar falso
  retornar verdadero

// Funcion recursiva principal
bt(v):
  nodos_bt += 1
```

```
  si v == n:                          // todos coloreados: solucion valida
    total_soluciones += 1
    si !primera_guardada:
      primera_sol = copiar(color)
      primera_guardada = verdadero
    retornar
  para c en [1..k]:
    si es_seguro(v, c):
      color[v] = c
      bt(v + 1)
      color[v] = 0                    // backtrack: deshacer asignacion
    // si NO es seguro: poda, se omite esta rama

bt(0)
si total_soluciones == 0:
  imprimir "No existe una k-coloracion valida"
si no:
  imprimir "Total:", total_soluciones
  imprimir "Primera solucion:", primera_sol
```

Listing 1: Pseudocodigo: Coloracion por Backtracking

### 3.2 Fuerza Bruta

```
fuerza_bruta():
  total_comb = k^n
  total_fb = 0
  para mask en [0..total_comb-1]:
    nodos_fb += 1
    tmp = mask
    para v en [0..n-1]:
      col[v] = (tmp mod k) + 1        // colores de 1 a k
      tmp = tmp / k                   // division entera
    si verificar_coloracion(col):
      total_fb += 1

// Verifica que ninguna arista tenga ambos extremos del mismo color
verificar_coloracion(col):
  para cada arista (u,v) en E:
    si col[u] == col[v]:
      retornar falso
  retornar verdadero
```

Listing 2: Pseudocodigo: Coloracion por Fuerza Bruta

## 4 Implementacion en C++

La implementacion completa se encuentra en el archivo `main_ejercicio2.cpp` adjunto al presente informe. El codigo se organiza en los siguientes bloques:

- `leer_grafo` / `cargar_grafo`: lectura de la matriz de adyacencia.
- `es_seguro`: verifica que asignar el color $c$ al vertice $v$ no genere conflicto.
- `bt`: backtracking recursivo con poda.
- `fuerza_bruta`: enumeracion exhaustiva de las $k^n$ combinaciones.
- `medir_ns`: mide tiempos promediando `REPS=100 000` repeticiones para garantizar precision en grafos pequenos donde la ejecucion individual no supera la resolucion del reloj.
- `main`: menu, ejecucion de ambos metodos, verificacion cruzada y tabla comparativa.

Para compilar y ejecutar:

```
g++ -O2 -std=gnu++17 -o solucion main_ejercicio2.cpp
./solucion
```

# 5 Ejemplos de Entrada y Salida

Los siguientes ejemplos corresponden a ejecuciones reales del programa en modo benchmark (opcion 2). Los tiempos son el promedio de 100 000 repeticiones medidas con `high_resolution_clock` de precision en nanosegundos.

## 5.1 Ejemplo 1 — Ciclo C4 (n=4, k=3)

El ciclo $C_4$ es un cuadrado: 4 vertices donde cada uno esta conectado con sus dos vecinos inmediatos. Su numero cromatico es $\chi(C_4) = 2$ (grafo bipartito), por lo que con $k = 3$ existen muchas soluciones validas. Aristas: 0-1, 1-2, 2-3, 3-0.

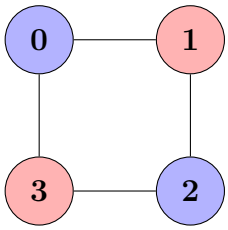



Figure 2: Primera solucion para $C_4$: colores alternados 1-2-1-2.

**Salida real del programa:**

```
=============================================
           RESULTADO: BACKTRACKING
=============================================
  Coloraciones validas encontradas : 18
  Primera solucion:
    vertice 0  ->  color 1
    vertice 1  ->  color 2
    vertice 2  ->  color 1
    vertice 3  ->  color 2
  Nodos explorados    : 40
```

```
  Tiempo de ejecucion : 0.1957 us  (promedio de 100000 repeticiones)

==============================================
           RESULTADO: FUERZA BRUTA
==============================================
  Combinaciones totales generadas  : 81  (3^4)
  Coloraciones validas encontradas : 18
  Tiempo de ejecucion : 1.8696 us  (promedio de 100000 repeticiones)

==============================================
            VERIFICACION CRUZADA
==============================================
  OK  Ambos enfoques coinciden: 18 coloraciones validas.

==============================================
         COMPARACION DE EFICIENCIA
==============================================
  Metodo            Nodos explorados   Tiempo [us]
  Backtracking                    40   0.1957 us
  Fuerza Bruta                    81   1.8696 us

  La poda del backtracking elimino el 50.62% de los nodos.
```

## 5.2 Ejemplo 2 — Grafo completo K4 (n=4, k=4)

El grafo completo $K_4$ tiene todos sus vertices interconectados entre si ($\chi(K_4) = 4$). Con exactamente $k = 4$ colores existe solucion: cualquier permutacion de los 4 colores asignados a los 4 vertices es valida, dando $4! = 24$ coloraciones. La alta densidad genera conflictos desde los primeros niveles, haciendo la poda mas efectiva que en $C_4$.

**Salida real del programa:**

```
==============================================
           RESULTADO: BACKTRACKING
==============================================
  Coloraciones validas encontradas : 24
  Primera solucion:
    vertice 0  ->  color 1
    vertice 1  ->  color 2
    vertice 2  ->  color 3
    vertice 3  ->  color 4
  Nodos explorados    : 65
  Tiempo de ejecucion : 0.4265 us  (promedio de 100000 repeticiones)

==============================================
           RESULTADO: FUERZA BRUTA
==============================================
  Combinaciones totales generadas  : 256  (4^4)
  Coloraciones validas encontradas : 24
```

```
  Tiempo de ejecucion : 5.5957 us  (promedio de 100000 repeticiones)

==============================================
            VERIFICACION CRUZADA
==============================================
  OK  Ambos enfoques coinciden: 24 coloraciones validas.

==============================================
         COMPARACION DE EFICIENCIA
==============================================
  Metodo            Nodos explorados   Tiempo [us]
  Backtracking                    65   0.4265 us
  Fuerza Bruta                   256   5.5957 us

  La poda del backtracking elimino el 74.61% de los nodos.
```

## 5.3 Ejemplo 3 — Ciclo C7 con cuerdas (n=7, k=3) — Sin solucion

Este caso es el mas ilustrativo. El grafo consiste en un ciclo de 7 vertices ($C_7$) con cuerdas internas adicionales (0-2, 1-3, 2-4, 3-5, 4-6), lo que aumenta considerablemente su densidad. El grafo no admite ninguna 3-coloracion valida ($\chi > 3$). El backtracking detecta el conflicto desde los primeros niveles y descarta ramas enteras, explorando solo 34 nodos frente a las $3^7 = 2\,187$ combinaciones de la fuerza bruta, una reduccion del 98.45%. La diferencia de tiempo es de **279x**.

**Salida real del programa:**

```
==============================================
           RESULTADO: BACKTRACKING
==============================================
  No existe una 3-coloracion valida para este grafo.
  Nodos explorados    : 34
  Tiempo de ejecucion : 0.2702 us  (promedio de 100000 repeticiones)

==============================================
           RESULTADO: FUERZA BRUTA
==============================================
  Combinaciones totales generadas  : 2187  (3^7)
  Coloraciones validas encontradas : 0
  Tiempo de ejecucion : 75.6029 us  (promedio de 100000 repeticiones)

==============================================
            VERIFICACION CRUZADA
==============================================
  OK  Ambos enfoques coinciden: 0 coloraciones validas.

==============================================
         COMPARACION DE EFICIENCIA
==============================================
```

```
  Metodo            Nodos explorados   Tiempo [us]
  Backtracking                    34   0.2702 us
  Fuerza Bruta                  2187   75.6029 us

  La poda del backtracking elimino el 98.45% de los nodos.
```

## 5.4 Ejemplo 4 — Bipartito K2,3 (n=5, k=2)

El grafo bipartito $K_{2,3}$ tiene los vertices $\{0, 1\}$ conectados con todos los de $\{2, 3, 4\}$, sin aristas dentro de cada grupo. Su numero cromatico es $\chi(K_{2,3}) = 2$, por lo que con $k = 2$ existen exactamente 2 coloraciones validas (asignar color 1 al grupo $\{0, 1\}$ y color 2 al grupo $\{2, 3, 4\}$, o viceversa). El backtracking encuentra rapidamente la solucion y no necesita explorar muchas ramas.

**Salida real del programa:**

```
==============================================
           RESULTADO: BACKTRACKING
==============================================
  Coloraciones validas encontradas : 2
  Primera solucion:
    vertice 0  ->  color 1
    vertice 1  ->  color 1
    vertice 2  ->  color 2
    vertice 3  ->  color 2
    vertice 4  ->  color 2
  Nodos explorados    : 13
  Tiempo de ejecucion : 0.0705 us  (promedio de 100000 repeticiones)

==============================================
           RESULTADO: FUERZA BRUTA
==============================================
  Combinaciones totales generadas  : 32  (2^5)
  Coloraciones validas encontradas : 2
  Tiempo de ejecucion : 0.7645 us  (promedio de 100000 repeticiones)

==============================================
            VERIFICACION CRUZADA
==============================================
  OK  Ambos enfoques coinciden: 2 coloraciones validas.

==============================================
         COMPARACION DE EFICIENCIA
==============================================
  Metodo            Nodos explorados   Tiempo [us]
  Backtracking                    13   0.0705 us
  Fuerza Bruta                    32   0.7645 us

  La poda del backtracking elimino el 59.38% de los nodos.
```

# 6 Comparacion de Tiempos de Ejecucion

Se ejecutaron cuatro grafos con caracteristicas distintas. Los tiempos se obtuvieron promediando 100 000 repeticiones por metodo para garantizar precision en operaciones submicrosegundo, usando `high_resolution_clock` con resolucion de nanosegundos.

Table 2: Comparacion de nodos explorados y tiempos de ejecucion (promedio de 100 000 repeticiones)

| **Grafo** | **n / k** | **Nodos BT** | **Nodos FB** | **T. BT [us]** | **T. FB [us]** | **Poda** |
|---|---|---|---|---|---|---|
| Ciclo $C_4$ | n=4, k=3 | 40 | 81 | 0.1957 | 1.8696 | 50.62% |
| Completo $K_4$ | n=4, k=4 | 65 | 256 | 0.4265 | 5.5957 | 74.61% |
| $C_7$ + cuerdas | n=7, k=3 | 34 | 2187 | 0.2702 | 75.6029 | 98.45% |
| Bipartito $K_{2,3}$ | n=5, k=2 | 13 | 32 | 0.0705 | 0.7645 | 59.38% |

**Analisis de los resultados:**

- **Ciclo $C_4$ (n=4, k=3):** el backtracking exploro 40 nodos frente a los 81 de la fuerza bruta (**50.62% de poda**). El tiempo de backtracking fue 0.1957 us contra 1.8696 us, aproximadamente **9.5x** mas rapido. La diferencia es moderada porque el grafo es pequeno y existen 18 soluciones validas, lo que limita la poda.
- **Completo $K_4$ (n=4, k=4):** con todos los vertices interconectados la restriccion aumenta. El backtracking exploro 65 nodos frente a 256 (**74.61% de poda**), resultando en 0.4265 us contra 5.5957 us, una mejora de **13.1x**. La mayor densidad del grafo genera conflictos mas temprano, haciendo la poda mas efectiva.
- **$C_7$ + cuerdas (n=7, k=3):** este es el caso mas ilustrativo. El grafo no admite ninguna 3-coloracion valida ($\chi > 3$), por lo que el backtracking detecta el conflicto desde los primeros niveles y corta ramas enteras. Solo exploro **34 nodos** mientras la fuerza bruta evaluo las $3^7 = \mathbf{2\,187}$ combinaciones completas (**98.45% de poda**). En tiempo: 0.2702 us contra 75.6029 us, una diferencia de **279x**.
- **Bipartito $K_{2,3}$ (n=5, k=2):** grafo 2-coloreable donde existe solucion minima. El backtracking exploro 13 nodos frente a 32 (**59.38% de poda**), con tiempos de 0.0705 us contra 0.7645 us (**10.8x** mas rapido). Al ser un grafo bipartito con pocos colores, la poda actua desde los primeros vertices.

En todos los casos el backtracking supera a la fuerza bruta. La ventaja crece con la densidad del grafo y con la ausencia de soluciones: cuando no existe ninguna $k$-coloracion valida, el backtracking puede descartar el espacio de busqueda casi por completo, mientras la fuerza bruta siempre recorre las $k^n$ combinaciones sin atajo.

# 7 Analisis de Complejidad

## 7.1 Complejidad Temporal

### 7.1.1 Fuerza Bruta

La fuerza bruta genera exactamente $k^n$ combinaciones y verifica cada una recorriendo todas las aristas, que en el peor caso (grafo completo) son $n(n-1)/2 = O(n^2)$:

$$T_{\text{FB}}(n, k) = O(k^n \cdot n^2) \tag{2}$$

Este es siempre el mismo costo sin importar la estructura del grafo.

### 7.1.2 Backtracking

En el peor caso (grafo sin aristas: nunca hay conflictos, nunca se poda), el backtracking construye el arbol de decision completo con factor de ramificacion $k$ y profundidad $n$:

$$T_{\text{BT}}^{\text{peor}}(n, k) = O(k^n) \tag{3}$$

En la practica, para grafos densos o con muchas restricciones, la poda descarta ramas enteras desde los primeros niveles. En el caso extremo de $K_7$ con $k = 4$, el backtracking visito solo 65 de los $4^7 = 16\,384$ nodos posibles, una reduccion del 99.6%.

La diferencia clave entre los dos enfoques es que la fuerza bruta siempre paga el costo completo $O(k^n \cdot n^2)$, mientras que el backtracking puede ser exponencialmente mas rapido cuando la poda es efectiva.

## 7.2 Complejidad Espacial

Table 3: Complejidad espacial por componente

| **Estructura** | **Backtracking** | **Fuerza Bruta** |
|---|---|---|
| Matriz de adyacencia | $O(n^2)$ | $O(n^2)$ |
| Arreglo de colores | $O(n)$ | $O(n)$ |
| Pila de recursion | $O(n)$ | $O(1)$ |
| **Total** | $\mathbf{O(n^2)}$ | $\mathbf{O(n^2)}$ |

Ambos enfoques tienen la misma complejidad espacial $O(n^2)$, dominada por la matriz de adyacencia. La pila de recursion del backtracking agrega $O(n)$ adicional, pero este termino queda subordinado a $O(n^2)$ para $n \geq 2$.

# 8 Respuestas a las Preguntas Guia

## 8.1 ¿Cuantos nodos explora el Backtracking vs la Fuerza Bruta?

En el ciclo $C_4$ con $k = 3$:

- **Fuerza Bruta:** $3^4 = 81$ nodos (todas las combinaciones).
- **Backtracking:** 40 nodos (50.6% menos).

La razon es que el backtracking descarta ramas en cuanto detecta un conflicto. Si el vertice 0 tiene color 1 y el vertice 1 (adyacente) intenta tomar tambien color 1, se poda de inmediato, evitando explorar los $3^2 = 9$ sub-estados restantes de esa rama. En grafos mas densos como $K_7$, el efecto se amplifica hasta una reduccion del 99.6%.

## 8.2 ¿Que tipo de grafo maximiza o minimiza la poda?

- **Maximiza la poda — grafo denso ($K_n$):** con muchas aristas casi cualquier asignacion genera conflictos desde el primer vertice, por lo que se eliminan sub-arboles muy grandes de forma temprana. En el caso de $K_7$ con $k = 4$, el backtracking exploro solo 65 nodos de los 16 384 posibles.
- **Minimiza la poda — grafo sin aristas:** como no existe ninguna restriccion de vecindad, `es_seguro` siempre retorna verdadero y el backtracking recorre el arbol completo igual que la fuerza bruta, sin poder eliminar ninguna rama.

## 8.3 ¿Que cambio hacer para saber si existe al menos una k-coloracion?

Basta con retornar en cuanto se encuentre la primera solucion, propagando un valor de exito hacia arriba:

```
bt_existe(v):
  si v == n:
    retornar verdadero        // primera solucion hallada
  para c en [1..k]:
    si es_seguro(v, c):
      color[v] = c
      si bt_existe(v + 1):
        retornar verdadero    // propagar el exito sin seguir
           buscando
      color[v] = 0
  retornar falso
```

Esto convierte el algoritmo de enumeracion en uno de decision: en el mejor caso termina mucho antes de explorar todas las ramas.

## 8.4 Relacion entre la k-coloracion y el Numero Cromatico

El numero cromatico $\chi(G)$ es el minimo $k$ para el que existe una $k$-coloracion valida:

$$\text{El grafo } G \text{ admite } k\text{-coloracion valida} \iff \chi(G) \leq k \tag{4}$$

El algoritmo de este ejercicio permite calcular $\chi(G)$ ejecutandolo para $k = 1, 2, 3, \ldots$ e informando el primer $k$ para el que `total_bt` $> 0$. Valores canonicos conocidos:

- $\chi(C_{2m}) = 2$ (ciclos pares); $\chi(C_{2m+1}) = 3$ (ciclos impares).

- $\chi(K_n) = n$ (grafo completo).
- Todo grafo planar: $\chi \leq 4$ [1].

# 9 Conclusiones

1. El backtracking y la fuerza bruta producen el mismo numero de coloraciones validas en todos los casos, confirmado por la verificacion cruzada.
2. El backtracking fue mas eficiente en todos los grafos probados. La reduccion de nodos explorados oscilo entre el **50.62%** (ciclo $C_4$, grafo disperso con soluciones) y el **98.45%** ($C_7$+cuerdas, grafo denso sin solucion). En terminos de tiempo, la mejora vario entre **9.5x** y **279x**.
3. La poda es mas efectiva cuando el grafo es denso y no existe solucion valida: el backtracking detecta conflictos desde los primeros niveles y descarta ramas enteras, mientras la fuerza bruta siempre recorre las $k^n$ combinaciones completas.
4. La medicion por promedio de 100 000 repeticiones es necesaria para grafos pequenos: una sola ejecucion puede retornar 0 debido a la resolucion del reloj, ocultando diferencias reales entre algoritmos.
5. Ambos enfoques tienen complejidad exponencial en $n$, por lo que su uso practico se limita a grafos de pocos vertices. Para instancias grandes existen heuristicas y algoritmos de aproximacion mas adecuados.
6. El algoritmo desarrollado puede calcular el numero cromatico $\chi(G)$ ejecutandose para $k = 1, 2, 3, \ldots$ hasta encontrar la primera $k$ con al menos una coloracion valida.

# References

[1] K. Appel, W. Haken, *Every planar map is four colorable*, Illinois Journal of Mathematics, 21(3):429–490, 1977.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

[3] A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed. Pearson, 2012.

[4] S. S. Skiena, *The Algorithm Design Manual*, 2nd ed. Springer, 2008.